Event*Sphere*: MongoDB Event Management System

Learning Reflection

Student ID: 664 870 797
Student Name: Chris Lawrence
Course: CSCI 485 - Topics in Computer Science (MongoDB/NoSQL)
Section: F25N01
Instructor: Dr. Kawal Jeet
Submission Date: November 30, 2025

Project Overview

Up to this point I had only ever worked with relational databases. Tables, joins, foreign keys—that's the world I knew, and then ORM frameworks like Django and SQLAlchemy. Switching to MongoDB felt like unlearning half of what I trusted. EventSphere is the project that made it click. Building an event management system from the ground up forced me to stop thinking in SQL and start thinking in terms of how data actually gets used. The more I worked with it, the clearer it became: NoSQL isn't a looser version of SQL. It solves problems that relational databases aren't built for.

Seeing MongoDB's Strengths in Practice

I walked into this course skeptical. SQL is predictable, you know what you're getting. I assumed MongoDB traded structure for flexibility in a way that would bite me later. That assumption didn't survive contact with the real features.

Geospatial queries hit first. Once I set up 2dsphere indexing and ran radius searches, I realized I was using the exact same tools that power driver lookups, event finders, and location-based recommendations. Not theoretical—immediately practical.

Text search was another shift. In SQL, full-text usually means bolting on application layer logic. Here, a few lines of code gave me weighted search ranking that worked exactly how we would expect a search engine to work. Simple and effective in a way SQL doesn't match for this kind of work.

Learning Through Real Constraints

The best lesson came from running into the 512MB Atlas quota when trying to push the free tier Atlas cluster to its limits. I'd created 47 indexes at one point, but later realized fewer were much better. Cutting that to 24 forced me to understand index cost, cache pressure, and when an index actually earns its place. It also made me think harder about balancing read speed, write speed, and storage—the kind of trade-off that matters in production.

The polymorphic pattern was another adjustment. In SQL, I'd split every event type into its own table. In MongoDB, storing different event shapes in a single collection turned out to be the right call. Real systems don't fit into rigid schemas. Flexibility isn't sloppiness here—it's a feature.

What Changed

This project shifted how I think about database design. I stopped seeing the database as "where the data goes" and started treating it as a core part of the system architecture. Patterns like the computed pattern showed me that doing work early can prevent expensive work later. Thinking in terms of read-patterns, data closeness, and document shape felt natural by the end.

There's more I want to explore, like transactions, change streams, GridFS, live analytics, and real-time updates. Unfortunately, we didn't get to cover MongoDB transactions in class, but I'm looking forward to learning how to implement multi-document ACID transactions for critical operations like ticket booking in future projects. The foundations are now in place, and that makes these advanced features easier to pick up down the road.

Reflection on the Process

EventSphere forced me to make decisions, not follow a checklist. I had to consider the bigger picture with scaling and performance in mind. Once I was thinking from the query first perspective, the project became much more manageable. The quota limit, indexing problems, and schema revisions were frustrating at first—but they're the parts that made the project meaningful.

Documenting the project helped more than I expected. Explaining *why* I chose extended references or computed fields made me understand them better. It helped me understand the trade-offs of different design choices.

Conclusion

EventSphere turned into more than an assignment. It became a full study of MongoDB's strengths, trade-offs, and performance characteristics. I came out understanding how document databases fit into modern application design and why so many large production systems rely on them.

More importantly, I proved to myself that I can design and optimize something that behaves like a real system, not just something that works in theory.

And I walk away with another portfolio project that I can use to showcase the system:

EventSphere Demo Website

Thank you for the opportunity to learn about MongoDB and NoSQL. I've enjoyed the course and learned a lot!

Key Technologies: MongoDB (Atlas), Python, Flask, JavaScript, HTML/CSS
Features Implemented: Geospatial queries, text search, analytics, polymorphic schema, strategic indexing, in depth documentation!